



com

[会员](#)

[商店](#)

[众包](#)

[新闻](#)

[博问](#)

[闪存](#)

[赞助商](#)

[Chat2DB](#)

[代码改变世界](#)



[注册](#)

[博客园](#)

[首页](#)

[新随笔](#)

[联系](#)

[订阅](#)

[管理](#)

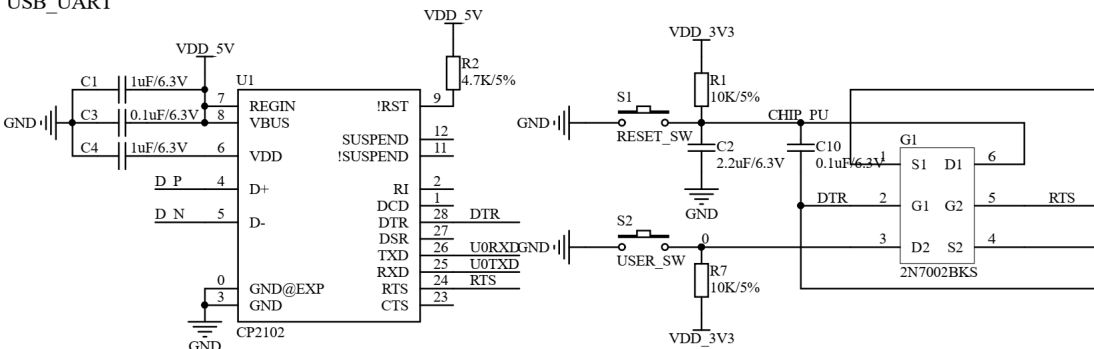
随笔 - 207 文章 - 0

ESP32-S3 通过USB Serial更新固件

本文主要介绍ESP32-S3在ubuntu20.04下通过ESP32-S3的USB Serial更新固件的方法以及遇到的问题的解决方法。

现在市面上ESP32-S3的开发板，大多都是ESP32-S3搭配一个USB-To-Serial桥芯片(CP210X、CH340等)来使用，硬件上再设计一个自动烧录电路(如下所示)，以达到通过串口实现固件更新的目的。

USB_UART



在实际的产品中，使用USB-To-Serial桥芯片不仅增加成本，也浪费PCB上面积。ESP32-S3是支持USB OTG的，可以使用这个USB口来进行固件更新吗？

当然可以！

除 USB OTG 之外，ESP32-S3 还内置一个 USB 串行/JTAG 控制器。当只使用内部收发器时，USB OTG 和 USB 串行/JTAG 外设共用这个收发器。默认情况下，内部收发器与USB 串行/JTAG 外设相连。当RTC_CNTL_SW_HW_USB_PHY_SEL_CFG 为 0 时，eFuse 中的 EFUSE_USB_PHY_SEL 位决定内部收发器与哪个外设相连。若该位为 0，内部收发器与 USB 串行/JTAG 外设相连；若该位为 1，内部收发器与 USB OTG 外设相连。当 RTC_CNTL_SW_HW_USB_PHY_SEL_CFG 为 1 时，由 RTC_CNTL_SW_USB_PHY_SEL_CFG控制内部收发器与哪个外设相连（与 EFUSE_USB_PHY_SEL 位的使用方式相同）。

CDC-ACM 接口遵循标准 USB CDC-ACM 类别进行虚拟串口通信，包含一个虚拟中断端点（不会发送任何事件，无使用需求）以及一个批量输入端点 (Bulk IN) 和批量输出端点 (Bulk OUT) 进行数据接收和发送。这些端点一次可以处理最高 64 字节的数据包，实现高吞吐量。CDC-ACM 为标准的 USB 设备类型，主机一般无需任何特殊安装程序就能正常工作，也就是说，当一个 USB 调试设备正确连接至主机时，操作系统应在片刻后显示新的串口信息。

除了通用的通信之外，CDC-ACM 接口还可以复位 ESP32-S3 并选择使其进入下载模式，从而烧录新的固件。这一功能可通过设置虚拟串口的 RTS 和 DTR 线来实现。

当我们的程序没有操作USB，完全按照eFuse的配置让USB工作以后，在Linux主机下看到的设备枚举信息如下：

```
usb 1-4: new full-speed USB device number 114 using xhci_hcd
usb 1-4: New USB device found, idVendor=303a, idProduct=1001, bcdDevice= 1.01
usb 1-4: New USB device strings: Mfr=1, Product=2, SerialNumber=3
usb 1-4: Product: USB JTAG/serial debug unit
usb 1-4: Manufacturer: Espressif
usb 1-4: SerialNumber: 48:27:E2:E1:E5:30
cdc_acm 1-4:1.0: ttyACM1: USB ACM device
```

这时我们使用esptool来更新固件是可以的，命令如下：

```
esptool --chip esp32s3 --port /dev/ttyACM1 --baud 921600 --before default_reset --after hard_reset write_flash -z .
```

注意：ubuntu20.04使用apt下载的esptool版本太低，不支持ESP32-S3的固件更新，需要到<https://github.com/espressif/esptool>下载最新的esptool。

但是当我们实现了ESP32-S3的USB Serial功能以后，就不能再使用esptool自动更新固件了，会一直打印connecting.....消息。需要先通过BOOT和Reset引脚让ESP32-S3进入下载模式，才能使用esptool更新固件成功。

USB Serial代码demo如下(基于Arduino IDE)：

```
#if ARDUINO_USB_MODE
#warning This sketch should be used when USB is in OTG mode

void setup() {}

void loop() {}

#else
#include "USB.h"
```

AI IDE Trae

公告

昵称： 闹闹爸爸
园龄： 6年10个月
粉丝： 32
关注： 46
+加关注

2025年3月			
日	一	二	三
23	24	25	26
2	3	4	5
9	10	11	12
16	17	18	19
23	24	25	26
30	31	1	2

搜索

常用链接

我的随笔
我的评论
我的参与
最新评论
我的标签

随笔分类

ACPI(21)
arm-linux(9)
Bluetooth(6)
C/C++(11)
Compile faults(7)
Device Tree(9)
Drivers(41)
EC(5)
Filesystem(1)
Hardware(4)
HDMI(1)
I2C/SMBus Subsystem(17)
IIO(1)
Intel(2)
J22440(7)
更多

随笔档案

2025年3月(29)
2025年2月(9)
2024年11月(2)
2024年10月(1)
2024年9月(6)
2024年8月(2)
2024年4月(3)
2024年3月(6)
2024年2月(3)
2024年1月(3)
2023年12月(4)
2023年11月(5)
2023年10月(3)
2023年4月(1)
2023年3月(6)
更多

阅读排行榜

1. aarch64-linux-gnu交叉编译C
2. 通过configs配置的Linux US
3. 解决：disagrees about versi
4. linux下WIFI模块使用：wpa_
5. shift count is too large(5751

评论排行榜

1. e1000e 0000:00:1f:6: The N
- Not Valid(8)

	2. linux下WIFI模块使用：wpa_ 又编译以及配置(8) 3. 使用STM8S i2c对TPS659871 4. I2C 10-bit 地址(3) 5. Failed to start discovery: org gress(1)
	推荐排行榜 1. 树莓派CM4 wifi频繁断开连接 2. 通过configfs配置的Linux US 3. 在用户空间实现I2C设备驱动 4. 交叉编译DHCP4.2.5：使得V P(2) 5. STM32 i2c通讯失败复位方法
	最新评论 1. Re:通过configfs配置的Linux 好文章 2. Re:e1000e 0000:00:1f:6: Th Is Not Valid @闹闹爸爸 好的，明白了，我 除命令，谢谢您... 3. Re:e1000e 0000:00:1f:6: Th Is Not Valid @999zzz 你那边网卡驱动应该i 如果要执行"/bootutil64e -NIC 网卡驱动卸载掉再执行。如果 卸载网卡驱动的命令是： rm... 4. Re:e1000e 0000:00:1f:6: Th Is Not Valid @闹闹爸爸 您的意思是我这边 载掉了吗... 5. Re:e1000e 0000:00:1f:6: Th Is Not Valid @999zzz 卸载I219网卡驱动： 行结果： ./bootutil64e -NIC 1 Connection to QV driver f...

```

#if ARDUINO_USB_CDC_ON_BOOT
#define HWSerial Serial0
#define USBSerial Serial
#else
#define HWSerial Serial
USBCDC USBSerial;
#endif

static void usbEventCallback(void* arg, esp_event_base_t event_base, int32_t event_id, void* event_data){
    if(event_base == ARDUINO_USB_EVENTS){
        arduino_usb_event_data_t * data = (arduino_usb_event_data_t*)event_data;
        switch (event_id){
            case ARDUINO_USB_STARTED_EVENT:
                HWSerial.println("USB PLUGGED");
                break;
            case ARDUINO_USB_STOPPED_EVENT:
                //HWSerial.println("USB UNPLUGGED");
                break;
            case ARDUINO_USB_SUSPEND_EVENT:
                HWSerial.printf("USB SUSPENDED: remote_wakeup_en: %u\n", data->suspend.remote_wakeup_en);
                break;
            case ARDUINO_USB_RESUME_EVENT:
                HWSerial.println("USB RESUMED");
                break;

            default:
                break;
        }
    } else if(event_base == ARDUINO_USB_CDC_EVENTS){
        arduino_usb_cdc_event_data_t * data = (arduino_usb_cdc_event_data_t*)event_data;
        switch (event_id){
            case ARDUINO_USB_CDC_CONNECTED_EVENT:
                HWSerial.println("CDC CONNECTED");
                break;
            case ARDUINO_USB_CDC_DISCONNECTED_EVENT:
                HWSerial.println("CDC DISCONNECTED");
                break;
            case ARDUINO_USB_CDC_LINE_STATE_EVENT:
                HWSerial.printf("CDC LINE STATE: dtr: %u, rts: %u\n", data->line_state.dtr, data->line_state.rts);
                break;
            case ARDUINO_USB_CDC_LINE_CODING_EVENT:
                HWSerial.printf("CDC LINE CODING: bit_rate: %u, data_bits: %u, stop_bits: %u, parity: %u\n", data->line_coding.bit_rate, data->line_coding.data_bits, data->line_coding.stop_bits, data->line_coding.parity);
                break;
            case ARDUINO_USB_CDC_RX_EVENT:
                HWSerial.printf("CDC RX [%u]:", data->rx.len);
                {
                    uint8_t buf[data->rx.len];
                    size_t len = USBSerial.read(buf, data->rx.len);
                    USBSerial.write(buf, data->rx.len); //write back to usb host
                    HWSerial.write(buf, len);
                }
                HWSerial.println();
                break;
            case ARDUINO_USB_CDC_RX_OVERFLOW_EVENT:
                HWSerial.printf("CDC RX Overflow of %d bytes", data->rx_overflow.dropped_bytes);
                break;

            default:
                break;
        }
    }
}

void setup() {
    HWSerial.begin(115200);
    HWSerial.setDebugOutput(true);

    USB.onEvent(usbEventCallback);
    USBSerial.onEvent(usbEventCallback);

    USBSerial.begin();

```


根据esp32-s3_technical_reference_manual_cn.pdf, 有如下信息:

(reserved)

Reset

-

AI IDE Trae

```

#else
#define HWSerial Serial
USBCDC USBSerial;
#endif

#define DR_REG_RTCCNTL_BASE          0x60008000
#define RTC_CNTL_USB_CONF_REG        (DR_REG_RTCCNTL_BASE + 0x120)

//write value to register
#define REG_WRITE(_r, _v) do {
    (*(volatile uint32_t *) (_r)) = (_v);
} while(0)

//read value from register
#define REG_READ(_r) ({
    (*(volatile uint32_t *) (_r));
})

void switch_to_download_mode(uint32_t ms)
{
    // 切换到USB Serial/JTAG控制器
    REG_WRITE(RTC_CNTL_USB_CONF_REG, 0x0);

    // 拉低D+和D-持续100ms,触发USB复位重新枚举
    REG_WRITE(RTC_CNTL_USB_CONF_REG, 0xa8);
    delay(ms);
    REG_WRITE(RTC_CNTL_USB_CONF_REG, 0x0);
}

static void usbEventCallback(void* arg, esp_event_base_t event_base, int32_t event_id, void* event_data){
    if(event_base == ARDUINO_USB_EVENTS){
        arduino_usb_event_data_t * data = (arduino_usb_event_data_t*)event_data;
        switch (event_id){
            case ARDUINO_USB_STARTED_EVENT:
                HWSerial.println("USB PLUGGED");
                break;
            case ARDUINO_USB_STOPPED_EVENT:
                //HWSerial.println("USB UNPLUGGED");
                break;
            case ARDUINO_USB_SUSPEND_EVENT:
                HWSerial.printf("USB SUSPENDED: remote_wakeup_en: %u\n", data->suspend.remote_wakeup_en);
                break;
            case ARDUINO_USB_RESUME_EVENT:
                HWSerial.println("USB RESUMED");
                break;

            default:
                break;
        }
    } else if(event_base == ARDUINO_USB_CDC_EVENTS){
        arduino_usb_cdc_event_data_t * data = (arduino_usb_cdc_event_data_t*)event_data;
        switch (event_id){
            case ARDUINO_USB_CDC_CONNECTED_EVENT:
                HWSerial.println("CDC CONNECTED");
                break;
            case ARDUINO_USB_CDC_DISCONNECTED_EVENT:
                HWSerial.println("CDC DISCONNECTED");
                break;
            case ARDUINO_USB_CDC_LINE_STATE_EVENT:
                HWSerial.printf("CDC LINE STATE: dtr: %u, rts: %u\n", data->line_state.dtr, data->line_state.rts);
                break;
            case ARDUINO_USB_CDC_LINE_CODING_EVENT:
                HWSerial.printf("CDC LINE CODING: bit_rate: %u, data_bits: %u, stop_bits: %u, parity: %u\n", data->line_coding.bit_rate, data->line_coding.data_bits, data->line_coding.stop_bits, data->line_coding.parity);
                break;
            case ARDUINO_USB_CDC_RX_EVENT:
                HWSerial.printf("CDC RX [%u]:", data->rx.len);
                {
                    uint8_t buf[data->rx.len];
                    size_t len = USBSerial.read(buf, data->rx.len);
                    USBSerial.write(buf, data->rx.len); //write back to host
                    HWSerial.write(buf, len);
                    if (buf[0] == 'U') { //如果收到'U'字符,切换到
                }
            }
        }
    }
}

```

```

        switch_to_download_mode(100);
    }
}

HWSerial.println();
break;
case ARDUINO_USB_CDC_RX_OVERFLOW_EVENT:
    HWSerial.printf("CDC RX Overflow of %d bytes", data->rx_overflow.dropped_bytes);
    break;

default:
    break;
}
}
}

void setup() {
    HWSerial.begin(115200);
    HWSerial.setDebugOutput(true);

    USB.onEvent(usbEventCallback);
    USBSerial.onEvent(usbEventCallback);

    USBSerial.begin();
    USB.begin();
}

void loop() {
    while(HWSerial.available()){
        size_t l = HWSerial.available();
        uint8_t b[l];
        l = HWSerial.read(b, l);
        USBSerial.write(b, l);
    }
}

#endif /* ARDUINO_USB_MODE */

```

如上述代码所示，当ESP32-S3 USB 收到'U'命令后，把USB切换到USB Serial/JTAG控制器，然后拉低D+和D-触发USB复位，重新枚举后即可直接使用esptool通过主机下CDC-ACM设备更新ESP32-S3的固件。

```

usb 1-4: new full-speed USB device number 119 using xhci_hcd
usb 1-4: New USB device found, idVendor=303a, idProduct=1001, bcdDevice= 1.00
usb 1-4: New USB device strings: Mfr=1, Product=2, SerialNumber=3
usb 1-4: Product: ESP32S3_DEV
usb 1-4: Manufacturer: Espressif Systems
usb 1-4: SerialNumber: 4827E2E1E530
cdc_acm 1-4:1.0: ttyACM1: USB ACM device // 以上是USB CDC-ACM模式枚举信息
usb 1-4: USB disconnect, device number 119 // ESP32-S3收到'U'命令, USB切换到USB Serial/JTAG控制器, 触发USB复位
usb 1-4: new full-speed USB device number 120 using xhci_hcd //以下是USB Serial/JTAG控制器模式枚举信息
usb 1-4: New USB device found, idVendor=303a, idProduct=1001, bcdDevice= 1.01
usb 1-4: New USB device strings: Mfr=1, Product=2, SerialNumber=3
usb 1-4: Product: USB JTAG/serial debug unit
usb 1-4: Manufacturer: Espressif
usb 1-4: SerialNumber: 48:27:E2:E1:E5:30
cdc_acm 1-4:1.0: ttyACM1: USB ACM device

```

本文来自博客园，作者：闹闹爸爸，转载请注明原文链接：<https://www.cnblogs.com/wanglouxiaozi/p/17987323>

分类: mcu

好文要顶

关注我

收藏该文

微信分享



闹闹爸爸
粉丝 - 32 关注 - 46

0

0

+加关注

升级成为会员

« 上一篇: [device descriptor read/64, error -11](#)

» 下一篇: [CVSD和mSBC的区别](#)

posted @ 2024-01-25 15:52 闹闹爸爸 阅读(2688) 评论(0) 编辑 收藏 举报

[刷新页面](#) [返回顶部](#)

登录后才能查看或发表评论, 立即 [登录](#) 或者 [逛逛](#) 博客园首页

【推荐】100%开源! 大型工业跨平台软件C++源码提供, 建模, 组态!

【推荐】还在用 ECharts 开发大屏? 试试这款永久免费的开源 BI 工具!

【推荐】国内首个AI IDE, 深度理解中文开发场景, 立即下载体验Trae

【推荐】编程新体验, 更懂你的AI, 立即体验豆包MarsCode编程助手

【推荐】轻量又高性能的 SSH 工具 IShell: AI 加持, 快人一步



相关博文:

- [Linux USB设备驱动\(二\)](#)
- [Linux USB设备驱动\(一\)](#)
- [翻译\[2\]-探讨一种自定义esp32-c3引导加载程序\(bootloader\)的方式](#)
- [stm32cubeide usb cdc hid](#)
- [stm32笔记\[16\]-使用usb-cdc串口.md](#)

阅读排行:

- [“你见过凌晨四点的洛杉矶吗?”--《我们为什么要睡觉》](#)
- [编程神器Trae: 当我用上后, 才知道自己的创造力被低估了多少](#)
- [C# 从零开始使用Layui.Wpf库开发WPF客户端](#)
- [C#/.NET/.NET Core技术前沿周刊 | 第 31 期 \(2025年3.17-3.23\)](#)
- [接口重试的7种常用方案!](#)